Elementos React

Programação Web 2

Objetivo de Aprendizagem

- Conhecer os elementos de uma aplicação React

Agenda

- Aplicação de demonstração Vite + React

Vite + React Demo

```
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <div>
        <a href="https://vitejs.dev" target="_blank">
          <img src={viteLogo} className="logo" alt="Vite logo" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img src={reactLogo} className="logo react" alt="React logo" />
        </a>
      </div>
      <h1>Vite + React</h1>
      <div className="card">
        <button onClick={() => setCount((count) => count + 1)}>
          count is {count}
        </button>
        <p>
          Edit <code>src/App.jsx</code> and save to test HMR
        </p>
      </div>
```

Importando Recursos

`useState`

```
1  import { useState } from 'react'
2  import reactLogo from './assets/react.svg'
3  import viteLogo from '/vite.svg'
4  import './App.css'
5
6  function App() {
7    const [count, setCount] = useState(0)
```

> O estado (*state*) permite gerenciar dados alterados em uma aplicação. Quando o *state* é modificado por alguma ação do usuário (clique, digitação, etc) a página é novamente renderizada

State

- Para a aplicação demo o estado consiste na variável `count` que representa a contagem de cliques no botão
- `setCount` é a função que modifica `count`
- `useState()` é um exemplo de *hook*. O estado é manipulado através de *hooks*
- `useState(0)` inicializa o estato inicial (variável `count`) com valor zero

Importando Recursos

- `/assets` recursos nesse diretório passam por otimização (*build*)

```
1  import { useState } from 'react'
2  import reactLogo from './assets/react.svg'
3  import viteLogo from '/vite.svg'
4  import './App.css'
```

React Functional Components

- Na aplicação demo temos um único componente `App()`
- Um componente React é apenas uma função JS que retorna JSX
- JSX é uma mistura de código JS e tags HTML

Tag Única

```
9   return (
10      <>
11        <div>
12          <a href="https://vitejs.dev" target="_blank">
13            <img src={viteLogo} className="logo" alt="Vite logo" />
14          </a>
15          <a href="https://react.dev" target="_blank">
16            <img src={reactLogo} className="logo react" alt="React logo" />
17          </a>
18        </div>
19        <h1>Vite + React</h1>
20        <div className="card">
21          <button onClick={() => setCount((count) => count + 1)}>
22            count is {count}
23          </button>
24          <p>
25            Edit <code>src/App.jsx</code> and save to test HMR
26          </p>
27        </div>
28        <p className="read-the-docs">
29          Click on the Vite and React logos to learn more
30        </p>
31      </>
32    )
```

Modificando *State*

- Observar a linha 13
- A interação do usuário (clique) altera *state*
- A cada clique o *event listener* associado ao botão é chamado(`setCount`)
- `setCount` modifica o estado o que forçando que a página seja renderizada novamente com *state* atualizado

```
11  <h1>Vite + React</h1>
12  <div className="card">
13    <button onClick={() => setCount((count) => count + 1)}>
14      count is {count}
15    </button>
16    <p>
```

Disponibilizando o Componente

`App`

- Observar a última linha
- Disponibiliza o componente de nome `App` para ser importado por `main.jsx` (responsável pela renderização)

```
1  export default App
```

Modificando o Componente

- Atualizar `App.jsx` como mostrado
- Observar que a página é renderizada logo após a modificação do arquivo

```
6  function App() {
7    return (
8      <div className="App">
9        <h1>Hello World!</h1>
10     </div>
11   );
12 }
13
14 export default App
```

Renderização

Renderização

`createRoot()`

- A grande funcionalidade do React é renderizar HTML
- A renderização acontece através do *container* `createRoot()`
- Tipicamente, é estruturado como uma `<div id="root"></div>` no `index.html`

index.html

Observar linhas 10-11

```
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6      <meta name="viewport" content="width=device-width, initial-scal
7      <title>Vite + React</title>
8    </head>
9    <body>
10     <div id="root">></div>
11     <script type="module" src="/src/main.jsx"></script>
12   </body>
13 </html>
```

main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

main.jsx

<StrictMode>

- Ferramenta puramente de desenvolvimento
- Destaca potenciais problemas no código
- Fornece *warnings*, etc

```
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import './index.css'
4  import App from './App.jsx'
5
6  createRoot(document.getElementById('root')).render(
7    <StrictMode>
8      <App />
```

`main.jsx`

`createRoot()`

- Busca `<div id="root">` no `index.html`
- A partir desse ponto a aplicação React é montada
- A renderização é efetivamente realizada por `.render(<App />)`

```
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import './index.css'
4  import App from './App.jsx'
5
6  createRoot(document.getElementById('root')).render(
7    <StrictMode>
8      <App />
```

`render()`

Conteúdo estático

- Define o que deve ser renderizado no *container* HTML
- Outro exemplo de `main.jsx`

```
1  import { createRoot } from 'react-dom/client'
2
3  createRoot(document.getElementById('root')).render(
4    <p>Welcome!</p>
5  )
```

`render()`

Exemplo

```
import { createRoot } from 'react-dom/client'

const myelement = (
  <table>
    <tr>
      <th>Name</th>
    </tr>
    <tr>
      <td>John</td>
    </tr>
    <tr>
      <td>Elsa</td>
    </tr>
  </table>
);

createRoot(document.getElementById('root')).render(
  myelement
)
```

Alterando o Nó Raiz

Root Node

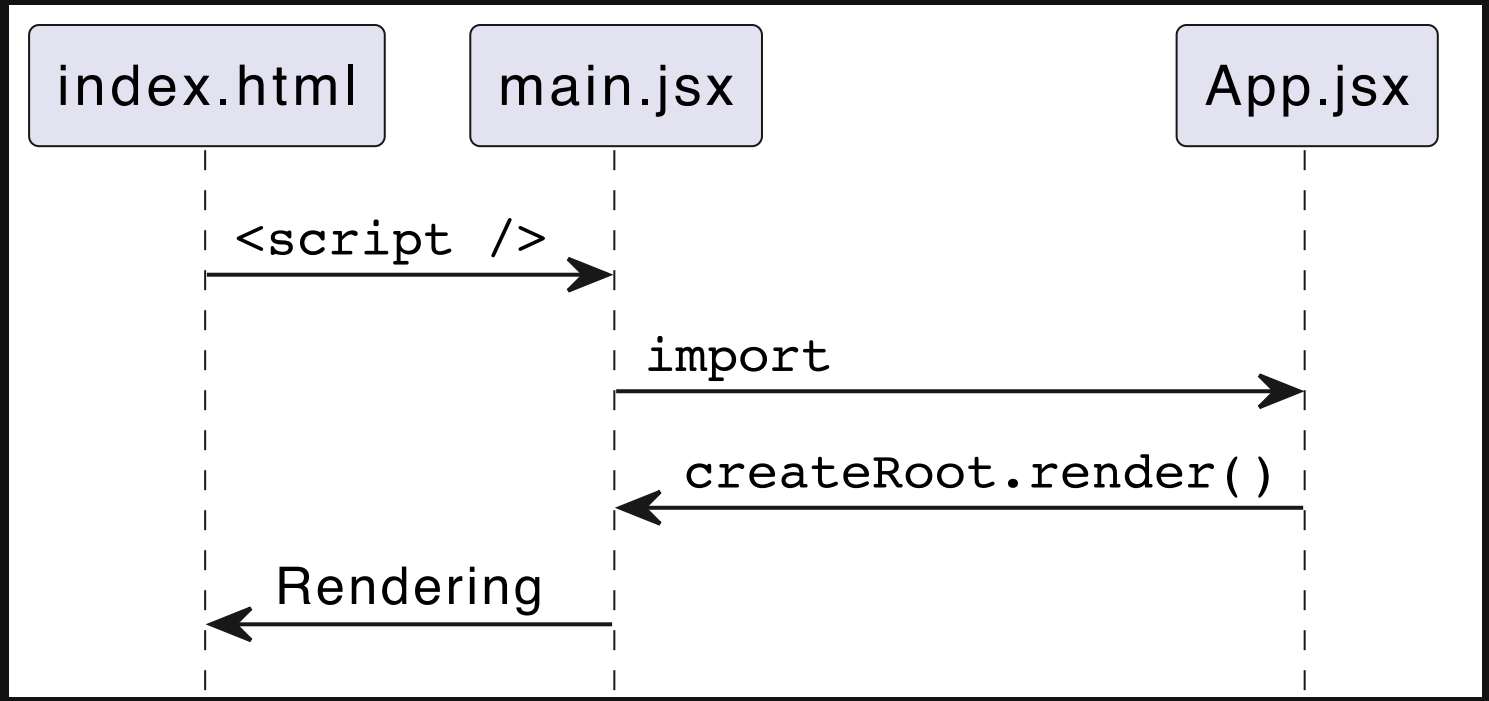
index.html

```html
<!doctype html>
<html lang="en">
  <body>
    <header id="sandy"></header>
    <script type="module"
    src="/src/main.jsx"></script>
  </body>
</html>
```

main.jsx

```jsx
import { createRoot } from 'react-dom/client'

createRoot(document.getElementById('sandy')).render(
  <p>Welcome!</p>
)
```


index.html
main.jsx
App.jsx
<script />
import
createRoot.render()
Rendering

Perguntas

Exercício

1

Crie uma aplicação React utilizando o Vite no seu ambiente local. Deixe a aplicação rodando. Modifique a aplicação como mostrado na aula. O que acontece? Foi preciso reiniciar o navegador?

Referências

- React Tutorial W3Schools
- React
- Deploy on Vercel

José Roberto Bezerra

jbroberto@ifce.edu.br

jbroberto76

Powered by Slidev

Cover image by haikei